

Data-Parallel Programming Language : C*



Presented by Yeh-Ching Chung

2002/09/13

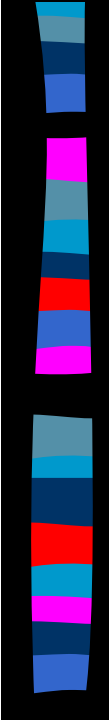
1

Outline

- Introduction
- A Simple C* Program
- Programming in C*
- C* Communication Functions
- Conclusions

2002/09/13

2

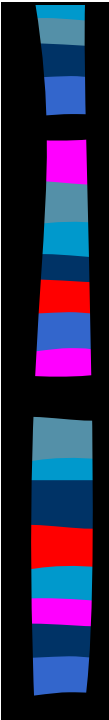


Introduction

- C* is an extension of the C programming language designed to help users program massively parallel distributed-memory computers.
 - A method for describing the size and shape of parallel data and for creating parallel variables.
 - New operators and expressions for parallel data, and new meanings for standard operators that allow them to work with parallel data.
 - Methods for choosing the parallel variables, and the specific data points within parallel variables, upon which C* code is to act.

2002/09/13

3



Introduction (Cont.)

- New kinds of points that point to parallel data and to shapes.
- Changes to the way functions work so that, for example, a parallel variable can be used an argument.
- Library function that also allow communication among parallel variable.
- Thinking Machines Corporation
 - May, 1993

2002/09/13

4

A Simple C* Program

```
#include <stdio.h>
shape [2][32768]ShapeA; /* step 1 */
int ShapeA p1, p2, p3;
int sum = 0;
main () {
    with (ShapeA) {
        p1=1; /* step 2 */
        p2=2; /* step 3 */
        P3=p1+p2; /* step 4 */
        printf("The sum in one element is %d.\n", [0][1]p3); /*step 5 */
        sum += p3; /* step 6 */
        printf("The sum in all elements is %d.\n", sum);
    }
}
```

2002/09/13

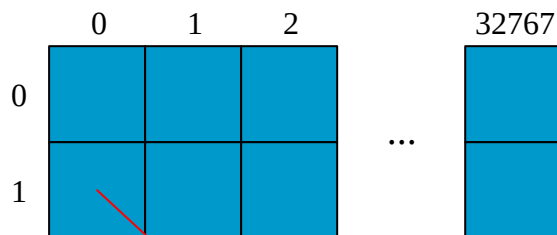
5

Step 1: Declaring Shapes Parallel Variables

■ Shapes

shape [2][32768]ShapeA;

ShapeA



[Back](#)

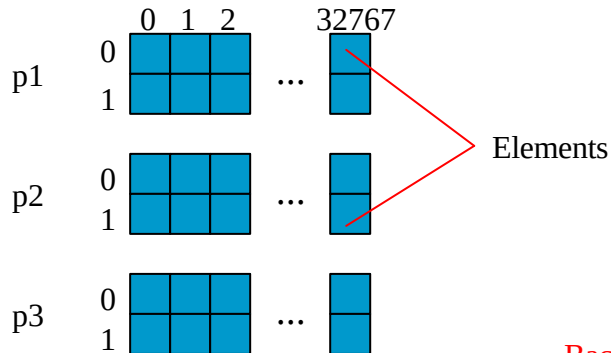
2002/09/13

6

Step 1: Declaring Shapes Parallel Variables (Cont.)

■ Parallel Variables

int:ShapeA p1, p2, p3;



[Back](#)

2002/09/13

7

Step 2: Select a Sheap

■ With Statement

– *current shape*

with (ShapeA)

[Back](#)

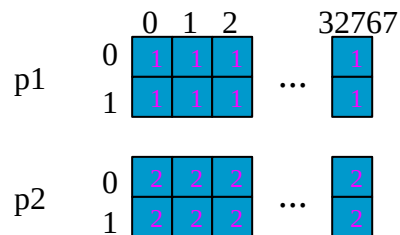
2002/09/13

8

Step 3: Assigning Values to Parallel Variables

P1 = 1;

P2 = 2;



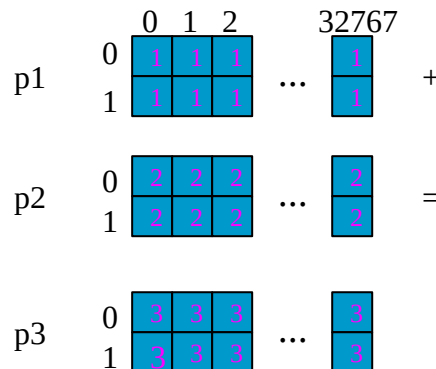
[Back](#)

2002/09/13

9

Step 4: Performing Computations Using Parallel Variables

P3 = p1 + p2;



[Back](#)

2002/09/13

10

Step 5: Choosing an Individual Element of a Parallel Variables

[0][1]p3

		0	1	2			32767
p3	0	3	3	3	...		3
	1	3	3	3			3

[Back](#)

2002/09/13

11

Step 6: Performing a Reduction Assignment of a Parallel Variables

sum += p3;

		0	1	2			32767
p3	0	3	3	3	...		3
	1	3	3	3			3

sum

196608

[Back](#)

2002/09/13

12

Programming in C*

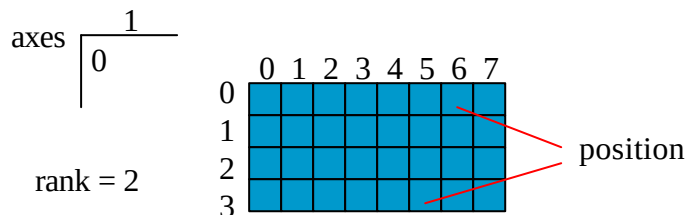
- Using Shapes and Parallel Variables
- Choosing a Shape
- Using C* Operators and Data Types
- Setting the Context
- Pointers
- Functions
- More on Shapes and Parallel Variables
- Communication

2002/09/13

13

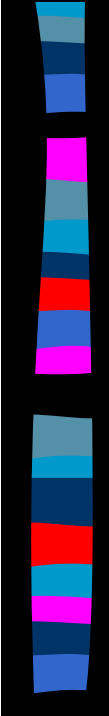
What is a Shape?

- A shape is a **template** for parallel data, a way of logically configuring data. You must define the shape of data before you operate on it.
 - The number of its dimensions: **rank**, **axis**
 - The number of **positions** in each of its dimensions



2002/09/13

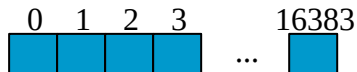
14



Declaring a Shape

- `shape [16384]employees;`
 - **shape** is a new keywords that C* adds to standard C.
 - **[16384]** specifies the number of positions in the shape.
 - **employees** is the number of the shape. Shape names follow standard C naming rules.

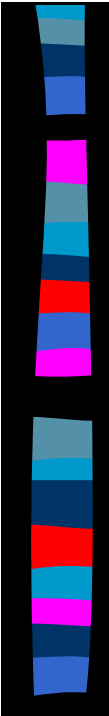
shape **employees**



- Declaring more than one shape
 - `shape [16384]employees, [256][512]images;`

2002/09/13

15



Obtaining Information about a Shape

- C* intrinsic function
 - **positionsof** takes a shape as an argument and returns the total number of positions in the shape.
 - **rankof** takes a shape as an argument and returns the shape's rank
 - **dimof** tanks two arguments: a shape and an axis number. It returns the number of positions along that axis.
 - Example: `shape[16384]employees, [256][512]image;`
 - `ranksof(employees)=1, positionsof(employees)=16384`
 - `ranksof(image)=2, positionsof(image)=131072`
 - `dimof(image, 0)=256, dimof(image, 1)=512`

2002/09/13

16

What is a Parallel Variable?

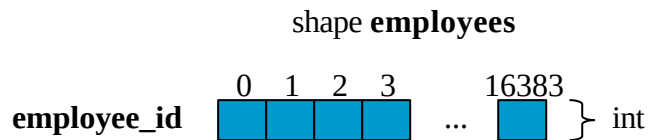
- Once a program has declared a shape, it can declare variables of that shape. These variables are called *parallel variables*.
- Parallel and Scalar Variables
 - Scalar variable
 - a type, along with its modifiers and qualifiers
 - a storage class that defines the manner in which the memory is to be allocated.
 - Parallel variable
 - Assign a constant to all elements of a parallel variable at the same time.
 - Perform an arithmetic operation on all elements of a parallel variable at the same time.
 - Do reduction assignments of data

2002/09/13

17

Declaring a Parallel Variable

- `unsigned int employee_id:employees;`



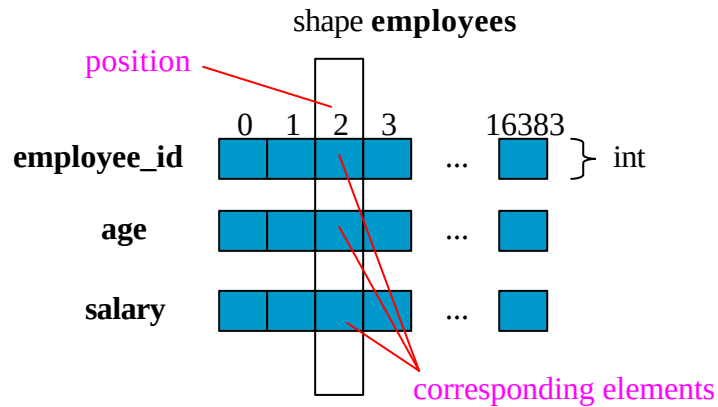
- Declaring more than one parallel variable
 - `unsigned int employee_id:employees, age: employees;`
 - `unsigned int employee_id:employees, field1:image;`
- A shortcut for declaring more than one parallel variable
 - `unsigned int: employees employee_id, age, salary;`
 - `int:shapeA p1, p2, p3;`

2002/09/13

18

Positions and Elements

- corresponding elements



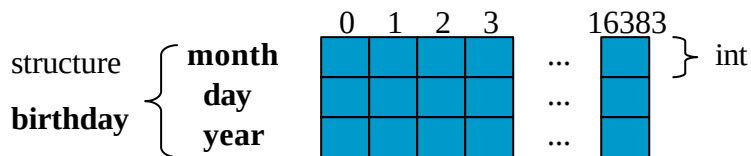
2002/09/13

19

Declaring a Parallel Structure

- Example:

```
shape [16384]employees;
struct date {
  int month,day, year ;
};
struct date: employees birthday;
shape employees
```



- Declaring more than one parallel structure

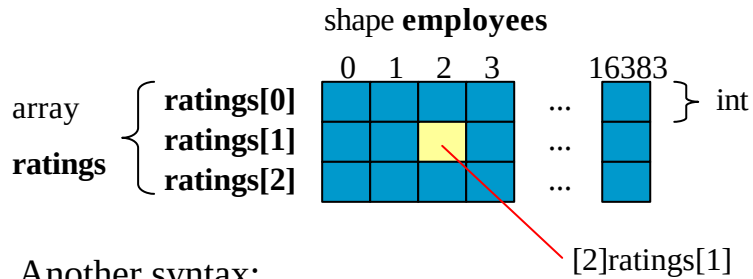
- struct date:employees birthday, date_of_hire;
- struct date birthday:employee, date_of_purch: equipment

2002/09/13

20

Declaring a Parallel Array

- Example: `int:employees ratings[3];`



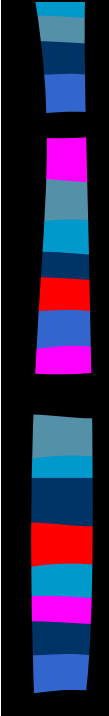
- Another syntax:
 - `int ratings[3]:employees;`

Initializing Parallel Variables

- The initializer must be a single scalar value.

- Examples

- `shape [65536]ShapeA;`
`int:ShapeA p1=6;`
- `main() {`
`int i=12;`
`shape [65536]ShapeA;`
`int:ShapeA p1=(6+i);`
`}`



Obtaining Information about Parallel Variables

■ C* intrinsic function

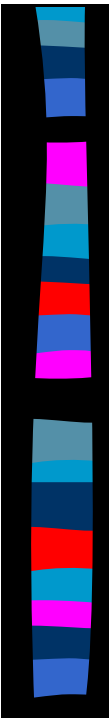
- **positionof**, **rankof**, **dimof**
- **shapeof** takes a parallel variable as an argument and returns the shape of the parallel variable. It is a *shape-valued expression*.

– Examples:

```
shape [16384]employees;    unsigned int:employees age;  
shapeof(age) returns the shape employees.  
unsigned int:employees salary => unsigned int:shapeof(age) salary  
struct date: employees birthday => struct date:shapeof(age) birthday
```

2002/09/13

23



Choosing a Shape

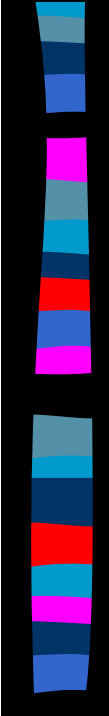
■ The **with** Statement

- A program can operate on parallel data from only one shape at a time. (*current* shape)
- We designate a shape to be the current shape by using the with statement.

```
shape [16384]employees;  
unsigned int:employees employee_id, age, salary, new_salary;  
with (employees) {  
    unsigned int sample_salary;  
    age = 0;  
    new_salary = salary * 2;  
    sample_salary = [0]salary;  
}
```

2002/09/13

24

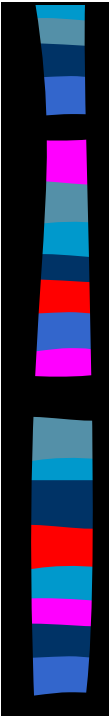


Choosing a Shape (Cont.)

```
shape [13864]employees, [8192]equipment;  
unsigned int employee_id: employees, date_of_purchase:equipment;  
main ( ) {  
    with (employees) {  
        date_of_purchase = 0;  
        [6]date_of_purchase = 0;  
    }  
}
```

2002/09/13

25

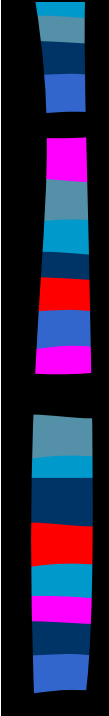


Using a Shape-Valued Expression

```
shape [13864]employees;  
unsigned int:employees age, salary;  
main ( ) {  
    with (shapeof(age)) {  
        salary = 200;  
    }  
}
```

2002/09/13

26



Nesting with Statement

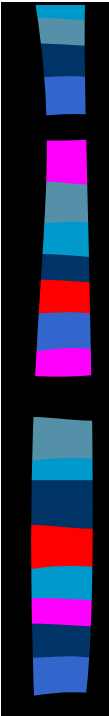
Ex1: with (employees) add_salaries();

Ex2: shape [16384]ShapeA, [32768]ShapeB;

```
int:ShapeA p1, p2; int:ShapeB q1;
main( ) {
    with (ShapeA) {
        p1 = 6;
        with (ShapeB) q1 = 12;
        p2 = 18;
    }
}
```

2002/09/13

27



Nesting with Statement (Cont.)

Ex3 :

```
with (ShapeA) {
    loop:
    with (ShapeB) {
        goto loop;
    }
}
```

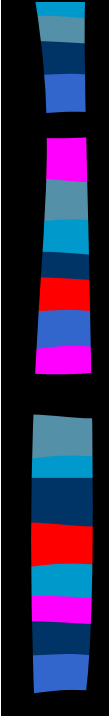
The **break**, **goto**, **continue**, and **return** statements also reset the current shape when they branch to an outer level.

Ex4:

```
goto loop;
with (ShapeA) {
    loop:
}
```

2002/09/13

28



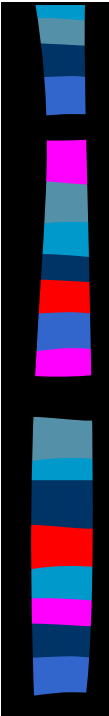
Initializing a Variable at Block Scope

Example:

```
shape [16384]ShapeA;  
int:ShapeA p1=6;  
main( ) {  
    with (ShapeA) {  
        int:ShapeA p2=p1;  
    }  
}
```

2002/09/13

29



Using C* Operators and Data Types

- C* uses all the Standard C operators, plus a few new operators of its own.
 - C* operators
 - C* a new datatype : *bool*
 - Parallel unions.

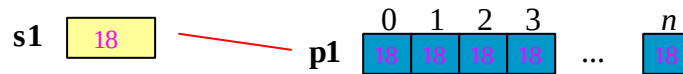
2002/09/13

30

With a Scalar and a Parallel Operand

■ Assignment with a Parallel LHS and a Scalar RHS

– $p1 = s1$



– $p1 + s1$

– $p1 == s1$

– $p1 \ll s1$

– $(p1 > 2) \&\& (s1 == 4)$

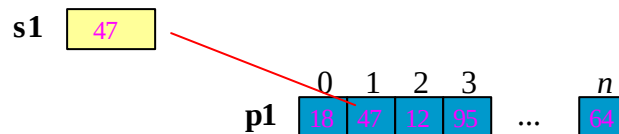
2002/09/13

31

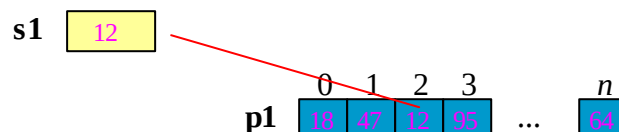
With a Scalar and a Parallel Operand

■ Assignment with a Scalar LHS and a parallel RHS

– $s1 = p1$ ✗ $\Rightarrow s1 = (\text{int})p1$ ✓



– $s1 = [2]p1$



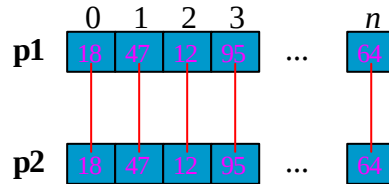
2002/09/13

32

With Two Parallel Operands

- Corresponding element

- Ex1: $p2 = p1$;



- Ex2: $p1 * p2$;

- Ex3: $p1 \geq p2$;

- Ex4: $(p1 > 2) \parallel (p2 < 4)$

2002/09/13

33

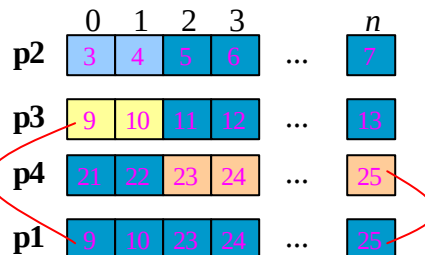
The Conditional Expression

- The ternary conditional expression $?$: operates in slightly different ways depending on mix of parallel and scalar variables in the expression.

– Ex1: $p1 = (s1 < 5) ? p2 : p3$;

– Ex2: $p1 = (s1 < 5) ? p2 : s2$;

– Ex3: $p1 = (p2 < 5) ? p3 : p4$;



2002/09/13

34

New C* Operators

■ The <? And >? Operators

- $a <? b ==> (a < b) ? a : b$
- Ex1: `s1 >?= s2;`
- Ex2: `p1 <?= p2;`

■ The %% Operator

- The operator provides the modulus of his operands.
- Ex1: `(17 %% 4) == 1`
- Ex2: `(17 %% -4) == -3`
- Ex3: `(-17 %% 4) == 3`
- Ex4: `(-17 %% -4) == -1`

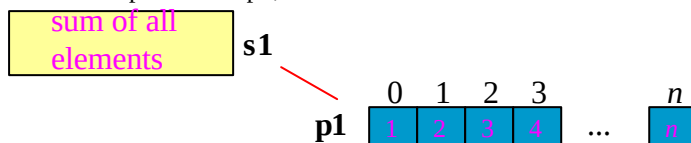
2002/09/13

35

Reduction Operators

■ C* reduction operators provide a quick way of performing operations on all elements of a parallel variable.

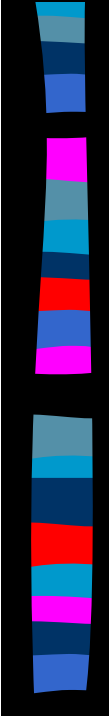
- Example : `s1 += p1;`



- Unary Reduction
 - Example: `printf("Total is %d\n", +=salary);`
- Parallel-to-Parallel Reduction Assignment
 - Example: `[0]payroll += salary;`

2002/09/13

36



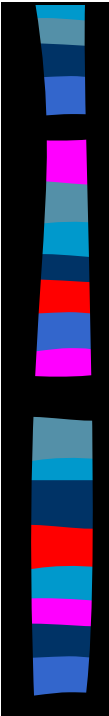
Reduction Operators (Cont.)

■ List of Reduction Operators

- += Sum of values of parallel variable elements
- -= Negative of the sum of values
- *= Product of values
- /= Reciprocal of the produce of value
- &= Bitwise AND of values
- ^= Bitwise XOR of values
- |= Bitwise OR of values
- <?= Minimum of values
- >?= Maximum of values
- %=, <<=, >>= can't be used as C* reduction operators.

2002/09/13

37



Reduction Operators (Cont.)

■ Minimum and Maximum Reduction Operators

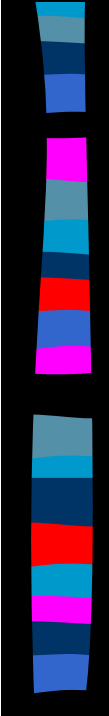
- Example 1:

```
printf("The lowest salary is %d\n", <?=salary);  
printf("The highest salary is %d\n", >?=salary);
```
- Example 2:

```
s1 <?= p1;  
s1 = <?=p1;
```

2002/09/13

38



The bool Data Type

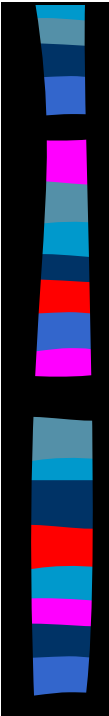
- The **bool** is a new unsigned integral data type in C*. The actual size and alignment of a bool are implementation-dependent.
 - bool is used to test conditions.
 - Example 1:

```
int i=0, j=4;
printf(“%d\n”, (bool)i);
printf(“%d\n”, (bool)j);
```
 - Example 2:

```
int i, j=1, k=1;
bool: current b;
i = j + k;
b = j + k;
```

2002/09/13

39



The bool Data Type (Cont.)

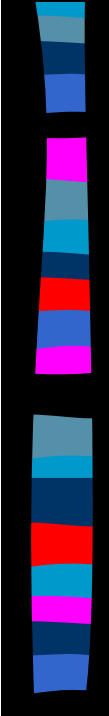
- The boolsizeof Operator
 - To obtain the exact size of a variable or data type in units of **bools**, use the new C* operator boolsizeof; boolsizeof has the same precedence and associativity as sizeof.
 - With a Parallel Variable or Data Type

```
boolsizeof(int:ShapeA);
```
 - With a Scalar Variable or Data Type

```
boolsizeof(int);
```

2002/09/13

40



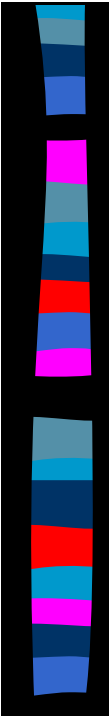
Parallel Unions

```
Union ptype {  
    int i;  
    float f;  
};  
union ptype:ShapeA p1;
```

```
p1.i = 50;  
p1.f = 89.7;
```

2002/09/13

41



Setting the Context

- What if we want an operation to be performed only on certain elements of a parallel variable?
- To do this, specify which positions are active by using a **where** statement. Using where to specify active positions it known as **setting the context**.

2002/09/13

42

The where Statement

- A where statement selects a subsets of these positions to remain active.

– Example :

```
with (population) {  
    where (weight > 150.0) {
```

```
    }
```

```
}
```

	0	1	2	3	4	5		32767
weight	148	109	170	212	116	222	...	151

2002/09/13

43

The where Statement (Cont.)

```
with (population) {  
    float avg_weight;  
    count = 1;  
    where (weight > 150.0)  
        avg_height = (+=height / +=count);  
        avg_weight = ( +=weight / +=count);  
}
```

2002/09/13

44

The else Statement

```
shape [32768]population;
float:population weight, height;
unsigned int:population count;
float avg_height_heavy, avg_height_light;
main () {
  with (population) {
    count = 1;
    where (weight > 150.0)
      avg_height_heavy = (+=height / +=count);
    else
      avg_height_light = (+=height / +=count);
  }
}
```

	0	1	2	3	4	5	...	32767
weight	148	109	170	212	116	222	...	151

2002/09/13

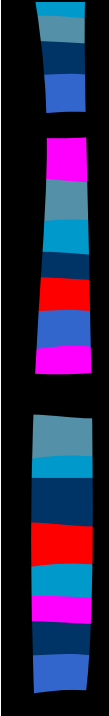
45

The where Statement and Scalar Code

```
shape [32768]population;
float:population weight, height;
unsigned int:population count;
main () {
  with (population) {
    count = 1;
    where (weight > 150.0) {
      [0]weight = 225;
      [0]weight = [1]weight;
      [0]count += count;
    }
  }
}
```

2002/09/13

46

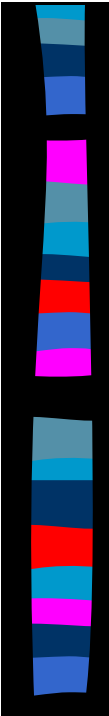


The everywhere Statement

```
shape [32768]population;
float:population weight, height;
unsigned int:population count, sex;
float avg_male_height, avg_female_height, avg_height;
main ( ) {
  with (population) {
    count = 1;
    where (weight > 150.0) {
      where (sex) avg_male_height = (+=height / +=count);
      else      avg_female_height = (+=height / +=count);
      everywhere avg_height = (+=height / +=count);
    }
  }
}
```

2002/09/13

47



Value of Unary Reduction Operators When There are No Active Positions

- += 0
- -= 0
- *= 1
- /= 1
- &= ~0
- ^= 0
- |= 0
- <?= maximum value representable
- >?= minimum value representable

2002/09/13

48

Preventing Code from Executing

```
if (|=(weight > 150.0))
  where (weight > 150.0) {
    float avg_height = 0;
  }
}
with (population) {
  unsigned int:population temporary = 0;
  if (|=(temporary = (++weight > 150.0))) {
    where (temporary) {
      float avg_height = 0;
    }
  }
}
```

2002/09/13

49

Context and the ||, &&, ?: Operators

■ p3 = (p1 > 5) && (p2++);

		0	1	2	3	4		32767
before	p1	1	7	-2	13	6	...	8
	p2	1	2	0	4	5	...	0
after	p3	0	1	0	1	1	...	0
	p2	1	3	0	5	6	...	1

2002/09/13

50

Context and the ||, &&, ?: Operators

■ `p3 = (p1 > 5) || (p2++);`

		0	1	2	3	4	...	32767
before	p1	1	7	-2	13	6	...	8
	p2	1	2	0	4	5	...	0
after	p3	1	1	0	1	1	...	1
	p2	2	2	1	4	5	...	0

2002/09/13

51

Pointers

- C* has three kinds of pointers
 - the Standard C pointer
 - a scalar pointer to a shape
 - a scalar pointer to a parallel variable
- Scalar-to-Scalar Pointers
 - `int *ptr` (declares ptr to be a scalar pointer to an int)
 - `ptr = &s1` (puts the address of s1 in ptr)
 - `s2 = *ptr` (puts the value of s1 into s2)

2002/09/13

52

Scalar Pointers to Shapes

■ Point to a shape

- `shape *ptr;` (declares the scalar variable `ptr` to be a pointer to a shape)
- `ptr = &ShapeA` (makes `ptr` point to `ShapeA`)
- `with (*ptr)` (makes `ShapeA` the current shape)

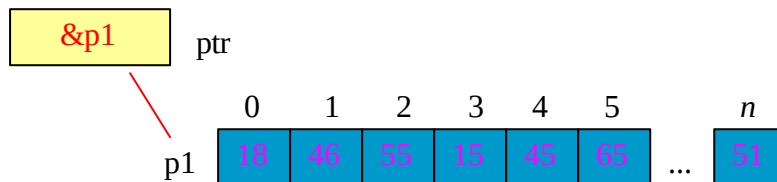
2002/09/13

53

Scalar Pointers to Parallel Variables

■ Point to a parallel variable

- `int:ShapeA *ptr` (declares a scalar pointer `ptr` that points to a parallel `int` of shape `ShapeA`)
- A pointer to a parallel variable in C* does not store a physical address, but a value that uniquely identifies the entire set of elements of the parallel variable.
- Ex1: `ptr = &p1;`

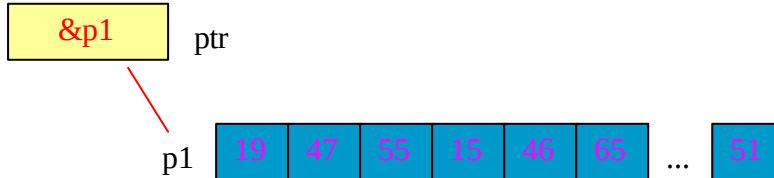


2002/09/13

54

Scalar Pointers to Parallel Variables

– Ex2: `(*ptr)++;`



– Ex3: `s1 += *ptr;`

Alternative Declaration Syntax

`int:ShapeA p1 => int:ShapeA *ptr;` ✓

`int p1:ShapeA => int *ptr:ShapeA;` ✗

– In this case, the compiler interprets the shape name as applying to the pointer, and parallel-to-scalar pointers do not exist in the language.

Pointer Arithmetic

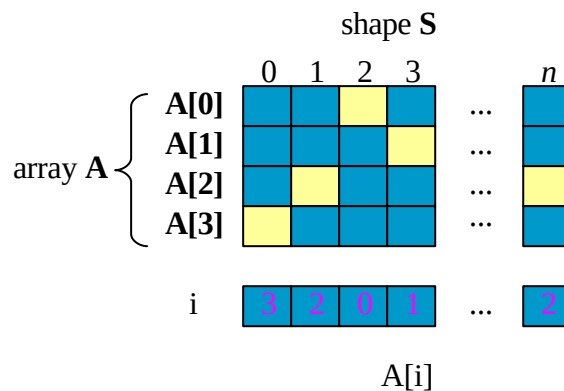
```
Shape [65536]ShapeA;  
int:ShapeA A1[40], *ptr1, *ptr2, p2, p3;  
  
ptr1=&A1[7];  
ptr2=ptr1 + 2;  
printf("%d\n", ptr2 - ptr1);  
  
p2 = *(A1+9);  
p3 = A1[9];  
  
ptr1 = &A1[7];  
ptr2 = ptr1 + p2;    => *ptr2 = *ptr1 + p2;  
p3 = *(ptr1 / p2);   => (*ptr1 / p2);
```

2002/09/13

57

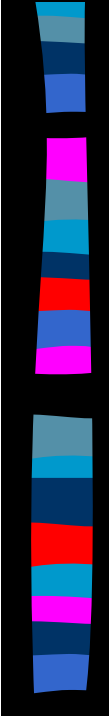
Parallel Indexes into Parallel Arrays

■ Parallel Right Indexing



2002/09/13

58

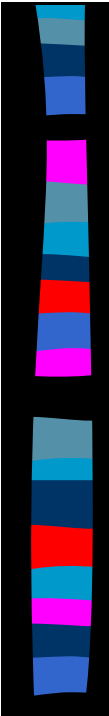


C* Pointers' Limitations

```
shape [8192]S;  
int:S A[4], i, p1, p2, *ptr;  
int s1;  
  
p1 = A[i];  
A[i] ++;  
p1 = *(A+i);  
p1 = *(ptr -i);  
  
s1 = &(A[i] );      /* Can't take the address */  
s1 = (A + i );      /* Creates invalid pointer type */  
p1 = ptr + p2;      /* Can't perform an operation without dereferencing */  
p1 = *(ptr / i);    /* Can only add or subtract */
```

2002/09/13

59

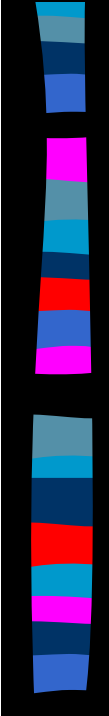


Functions

- C* add support for parallel variables and shapes to standard C functions
 - C* functions can take parallel variables and shapes as arguments.
 - C* functions can return parallel variables and shapes.
 - C* adds a new keyword **current**, which you can use to specify that a variable is of the current shape.
 - C* includes a **void** predeclared shape name so that you can declare an argument to be a pointer to a parallel variable of any shape.
 - C* supports overloading of functions, so that functions operating on scalar and on parallel data can have the same name.

2002/09/13

60



Passing a Parallel Variable as an Argument

```
void print_num(int:ShapeA x) {  
    printf("The sum is %d\n", +=x);  
}
```

If p1 is a parallel variable of type int and shape ShapeA
print_sum(p1);

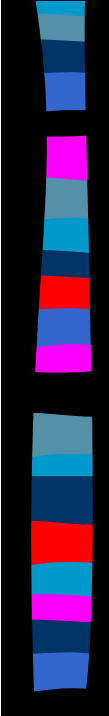
If the Parallel Variable is not of the Current Shape

```
void print_num(int:ShapeA *x) {  
    with (ShapeA)  
        printf("The sum is %d\n", +=*x);  
}
```

If p1 is a parallel variable of type int and shape ShapeA
print_sum(&p1);

2002/09/13

61



Returning a Parallel Variable

```
float:ShapeA increment(float:ShapeA x) {  
    return (x + 1.);  
}
```

Assume that p1 and p2 are parallel floats of shape ShapeA, and ShapeA is the current shape.

```
p2 = increment(p1);
```

The header of the function increment

```
float increment(float:ShapeA x):ShapeA
```

2002/09/13

62

In a Nested Context

```
float:ShapeA increment_if_over_5(float:Shape A x, float:Shape A y) {  
  where ( y > 5. ) return (x + 1.);  
}  
with (ShapeA) p3 = increment_if_over_5( p1, p2);
```

shape **ShapeA**

	0	1	2	3	4	5	...	<i>n</i>
p1	0.0	1.0	2.0	3.0	4.0	5.0	...	6.0
p2	5.4	4.9	4.2	1.2	7.8	6.1	...	1.0
p3	1.0				5.0	6.0	...	

2002/09/13

63

Passing a Shape as an Argument

```
int number_of_active_positions(shape x) {  
  with (x) {  
    int:x local = 1;  
    return ( += local);  
  }  
}
```

The shape that you pass need not be the current shape.

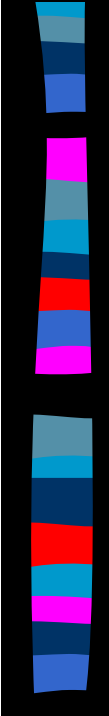
```
float raise(shape employees, float:employees salary):employees {  
  return (1.1 * salary);  
}
```

employees must be the current shape.

You pass more than one shape, and data is passing between the shapes.

2002/09/13

64

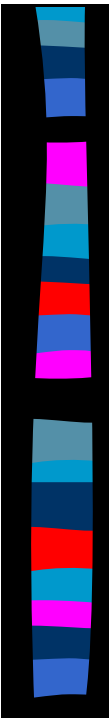


Returning a Shape

```
Shape choose_shape(shape ShapeA, shape ShapeB, int n) {  
    if (n) return ShapeA;  
    else return ShapeB;  
}
```

This function returns ShapeA or ShapeB, depending on the value of n.

```
with(choose_shape(shape1, shape2, s1))
```

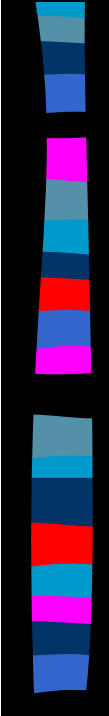


The current Predeclared Shape Name

int:current variable1;

if **employees** is the current shape when this statement is executed, **variable1** is of shape **employees**; if **image** is the current shape, **variable1** is of the shape **image**.

```
void print_sum(int:current x) {  
    printf("The sum is %d\n", +=x);  
}
```

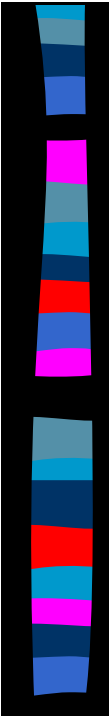


The void Predeclared Shape Name

```
int sum(int:void *x) {  
    with (sizeof(*x))  
        return (+= *x);  
}
```

2002/09/13

67



Using sizeof with the void Shape

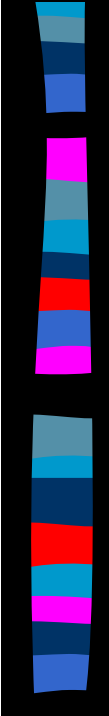
```
int sum_of_two_vars(int:void *x, int:sizeof(*x) *y) {  
    with (sizeof(*x))  
        return (+= (*x + *y));  
}
```

For parameters declared locally within the function, use **current**:

```
float average(int:void *x) {  
    with(sizeof(*x)) {  
        int:current y = 1;  
        return ( +=*x / +=y);  
    }  
}
```

2002/09/13

68



Overloading Function

- It maybe convenient for you to have more than one version of a function with the same name.

```
void f(int x);
```

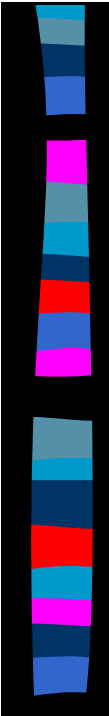
```
void f(int x, int y);
```

```
void f(int:current x);
```

```
overload f;
```

2002/09/13

69

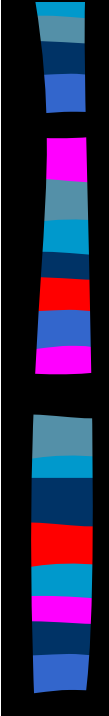


More on Shapes and Parallel Variable

- Partially specifying a shape
- Creating copies of shapes
- Dynamically allocating and deallocating a shape.
- Using the C* library function **palloc** to explicitly allocate storage for parallel variable
- Casting to shape, and casting to or from a parallel data type.

2002/09/13

70

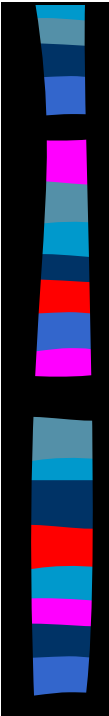


Partially Specifying a Shape

- To declare a shape without fully specifying its rank and dimensions.
 - `shape ShapeA;`
 - `shape [ ]ShapeB;`
- **rankof** intrinsic function
 - `shape s, [ ]t, [8092]u;`
`rankof(s) == 0;`
`rankof(t) == 2;`
`rankof(u) == 1;`

2002/09/13

71

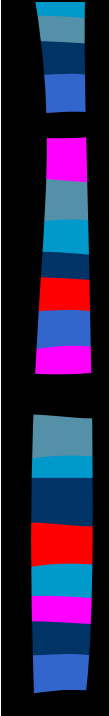


Partially Specifying a Shape (Cont.)

- Partially Specifying an Array of Shapes
 - `shape ShapeC[10];`
 - `shape [ ][ ]ShapeD[10];`
- Arrays and Pointers
 - `shape *ptr; ptr= Sarray;`
`Sarray[3] == *(ptr +3) == *(Sarray + 3)`
- Limitations
 - `shape [ ][4]ShapeE;` ✗
 - `shape [ ]ShapeF;` ✓

2002/09/13

72

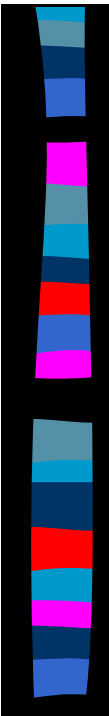


Creating Copies

- One way to fully specify a shape is by using the assignment operator to copy a fully specified shape to a partially specified one.
 - Shape ShapeA;
Shape [256][256]ShapeB;
ShapeA = ShapeB;

2002/09/13

73



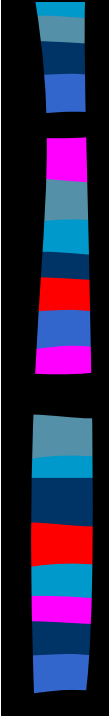
Assigning a Local Shape to a Global Shape

```
shape ShapeA;  
void f(void) {  
    shape [1024][512]ShapeB;  
    shapeA = ShapeB;  
}  
main {  
    f();  
    {  
        int:ShapeA p1;  
    }  
}
```

Don't Assign a Local Shape to a
Global Shape

2002/09/13

74

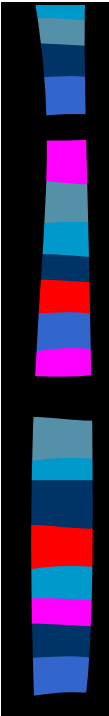


Dynamically Allocating a Shape

- C* intrinsic function - ***allocate_shape***
 - shape []ShapeB;
shapeB = allocate_shape(&ShapeB, 1, 65536);
 - allocate_shape(&new_shape, 3, 2, 2, 4096);
 - ShapeD[0] = allocate_shape(&ShapeD[0], 2, 4, 16384);

2002/09/13

75



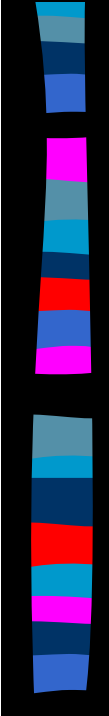
Deallocating a Shape

- C* intrinsic function - ***deallocate_shape***

```
#include <stdlib.h>
shape [ ]S;
int positions = 4096;
main( ) {
    while(positions <= 65536) {
        S = allocate_shape(&S, 1, positions);
        int:S p1, p2, p3;
        deallocate_shape(&S);
        positions *= 2;
    }
}
```

2002/09/13

76



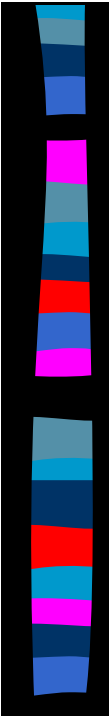
Dynamically Allocating a Parallel Variable

- C* intrinsic function - *palloc*, *pfree*

```
#include <stdlib.h>
shape [16384]ShapeA;
int:ShapeA *ptr;
main( ) {
    ptr= palloc(ShapeA, boolsizeof(int:ShapeA));
    pfree(ptr);
}
```

2002/09/13

77

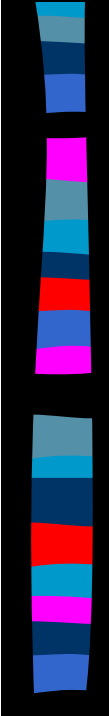


Dynamically Allocating a Parallel Variable (Cont.)

```
#include <stdio.h>
shape S;
double:S *p;
main( ) {
    S = allocate_shape(&S, 2, 4, 8192);
    p = palloc(S, boolsizeof(double:S));
    /* ... */
    pfree(p);
    deallocate_shape(&S);
}
```

2002/09/13

78

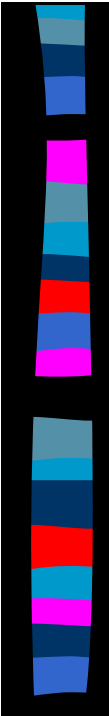


Casting with Shape and Parallel Variable

- Use the C* cast operator to cast an expression to a particular shape and type.
- Scalar-to-Parallel Casts
 - `s1 = +=(int:current)1;`
- Parallel-to-Parallel Casts
 - Casts to different type
 - `int:ShapeA p1; sqrt((double:ShapeA)p1);`

2002/09/13

79

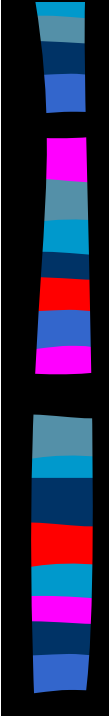


Casting with Shape and Parallel Variable (Cont.)

- Parallel-to-Parallel Casts (Cont.)
 - Casts to a different shape
- ```
shape [22256][256]ShapeB, ShapeA;
main () {
 ShapeA = ShapeB;
 int a:ShapeA, b:ShapeB;
 with(ShapeB) {
 b = a;
 b = (int:ShapeB)a;
 }
}
```

2002/09/13

80

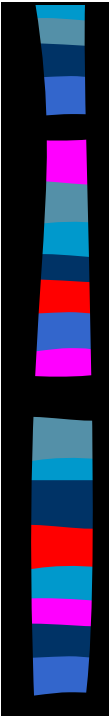


## The physical Shape

- **physical** is a new keyword that C\* adds to C. The shape **physical** is always of rank 1, its number of positions is the number of physical processors in which your program is running.
  - (int:physical) p1;

2002/09/13

81



## Communication

- Use to perform communication among parallel data.
  - Sending values of parallel variable elements to other element of the same or a different shape.
  - Getting values of parallel variable elements that are of the same or a different shape.
- C\* provides two methods of communication
  - General Communication
  - Grid Communication

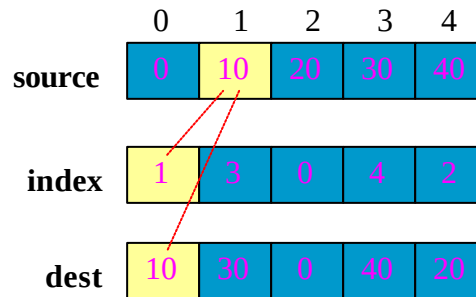
2002/09/13

82

## Using a Parallel Left Index for a Parallel Variable

### ■ A Get Operation

–  $\text{dest} = [\text{index}]\text{source};$



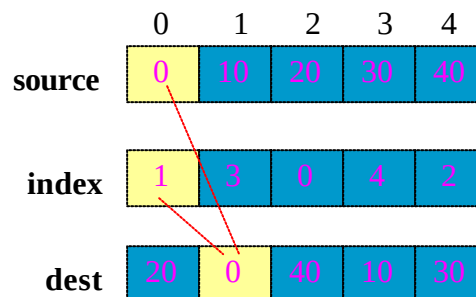
2002/09/13

83

## Using a Parallel Left Index for a Parallel Variable (Cont.)

### ■ A Send Operation

–  $[\text{index}]\text{dest} = \text{source};$

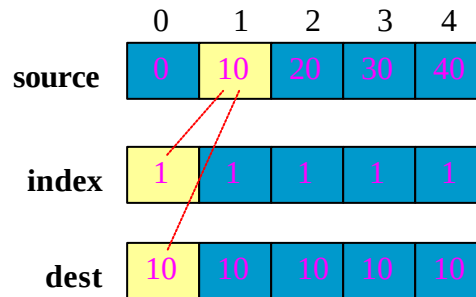


2002/09/13

84

## When There Are Potential Collisions

- For a Get Operation
  - $\text{dest} = [\text{index}]\text{source};$

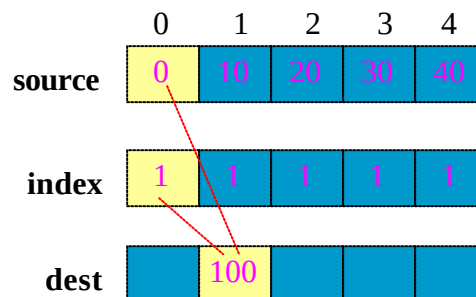


2002/09/13

85

## When There Are Potential Collisions

- For a Send Operation
  - $[\text{index}]\text{dest} += \text{source};;$



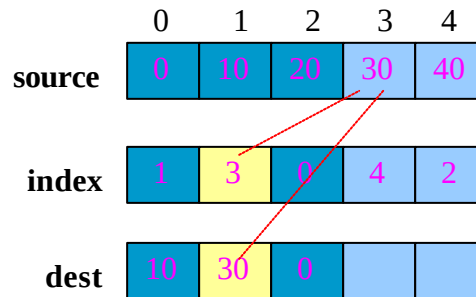
2002/09/13

86

## When There Are Inactive Position

### ■ For a Get Operation

– where  $(source < 30)$   $dest[index] = source;$



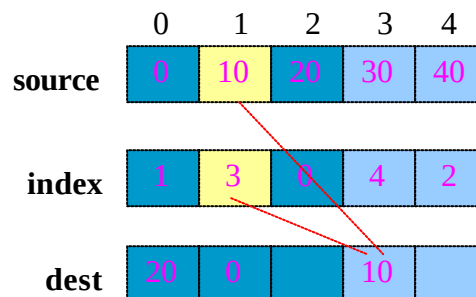
2002/09/13

87

## When There Are Inactive Position

### ■ For a Send Operation

– where  $(source < 30)$   $[index]dest = source;$



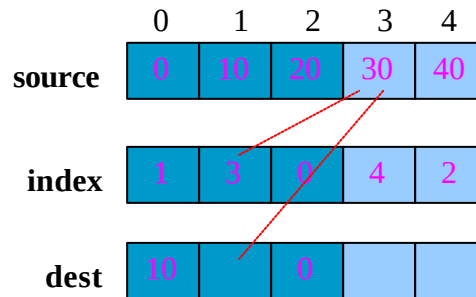
2002/09/13

88

## When There Are Inactive Position

### ■ Send and Get Operation in Function Call

- `int:current get_op(int:current source, int:current index)`  
`{ return ([index]source); }`



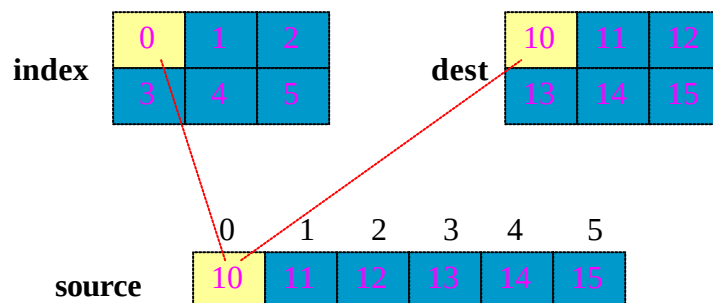
2002/09/13

89

## Mapping a Parallel Variable to Another Shape

### ■ For a Get Operation

- `dest = [index]source;`



2002/09/13

90

## An Example: Add Diagonals in a Matrix

```
shape [4][4]ShapeA;
shape [7]ShapeB;
int:ShapeA source, index;
int:ShapeB dest=0;
main () {
 with(ShapeA)
 [index]dest += source;
}
```

2002/09/13

91

## An Example: Add Diagonals in a Matrix (Cont.)

With (ShapeA) [index]dest +=source;

|        |    |    |    |    |       |   |   |   |   |
|--------|----|----|----|----|-------|---|---|---|---|
| source | 0  | 1  | 2  | 3  | index | 3 | 4 | 5 | 6 |
|        | 4  | 5  | 6  | 7  |       | 2 | 3 | 4 | 5 |
|        | 8  | 9  | 10 | 11 |       | 1 | 2 | 3 | 4 |
|        | 12 | 13 | 14 | 15 |       | 0 | 1 | 2 | 3 |
|        |    |    |    |    |       |   |   |   |   |
| dest   | 0  | 1  | 2  | 3  | 4     | 5 | 6 |   |   |
|        | 12 | 21 | 27 | 30 | 18    | 9 | 3 |   |   |

2002/09/13

92

## Using the pcoord Function

- Using pcoord to create a parallel variable in the current shape; each element in the variable is initialized to its coordinate along the axis you specify as the argument to pcoord.
  - p1 = pcoord(0);

|    | 0 | 1 | 2 | 3 | 4 | 5 |     | 32767 |
|----|---|---|---|---|---|---|-----|-------|
| p1 | 0 | 1 | 2 | 3 | 4 | 5 | ... | 32767 |

2002/09/13

93

## Using the pcoord Function (Cont.)

shape [4][4096]ShapeB;

int:ShapeB p2;

main ( ) {

    with (ShapeB)     p2 =pcoord(1);

}

|   | 0 | 1 | 2 | 3 | 4 | 5 |     | 4095 |
|---|---|---|---|---|---|---|-----|------|
| 0 | 0 | 1 | 2 | 3 | 4 | 5 | ... | 4095 |
| 1 | 0 | 1 | 2 | 3 | 4 | 5 | ... | 4095 |
| 2 | 0 | 1 | 2 | 3 | 4 | 5 | ... | 4095 |
| 3 | 0 | 1 | 2 | 3 | 4 | 5 | ... | 4095 |

2002/09/13

94

## Using the pcoord Function (Cont.)

```
shape [4][4096]ShapeB;
```

```
int:ShapeB p2;
```

```
main () {
```

```
 with (ShapeB) p2 =pcoord(1);
```

```
}
```

|   | 0 | 1 | 2 | 3 | 4 | 5 |     | 4095 |
|---|---|---|---|---|---|---|-----|------|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 0    |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | ... | 1    |
| 2 | 2 | 2 | 2 | 2 | 2 | 2 | ... | 2    |
| 3 | 3 | 3 | 3 | 3 | 3 | 3 | ... | 3    |

2002/09/13

95

## An Example

### ■ Matrix Transposition

```
shape [3][3]ShapeA;
```

```
int:ShapeA matrix, mew_matrix;
```

```
main() {
```

```
 with (ShapeA) [pcoord(1)][pcoord(0)]new_matrix = matrix;
```

```
}
```

**matrix**

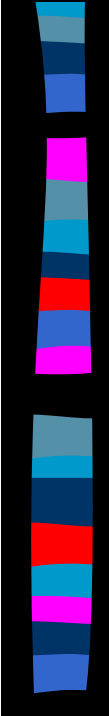
|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

**New\_matrix**

|   |   |   |
|---|---|---|
| 0 | 3 | 6 |
| 1 | 4 | 7 |
| 2 | 5 | 8 |

2002/09/13

96

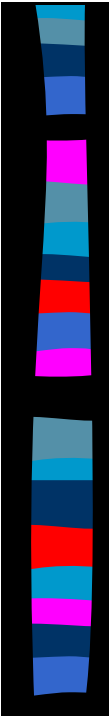


## The pcoord Function and Grid Communication

- Get Communication
  - `dest = [pcoord(0)+1]source;`
- Send Communication
  - `[pcoord(0)+1]dest = source;`
- Multi-dimension Communication
  - `dest = [pcoord(0)-2][pcoord(1)+1]source;`
  - `dest = [ . -2][ . +1]source;`

2002/09/13

97

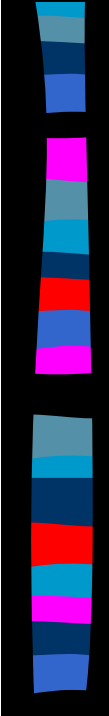


## The pcoord Function and Grid Communication (Cont.)

- Grid Communication without Wrapping
  - where `(pcoord(0) > 1)`  
`dest = [ . -2]source;`
  - where `(pcoord(0) < (dimof(ShapeA, 0) - 2))`  
`dest = [ . +2]source;`

2002/09/13

98



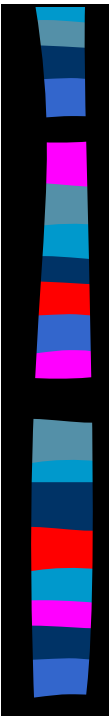
## The pcoord Function and Grid Communication (Cont.)

### ■ Grid Communication with Wrapping

```
dest = [(.+ 2) %% dimof(ShapeA, 0)]source;
```

2002/09/13

99



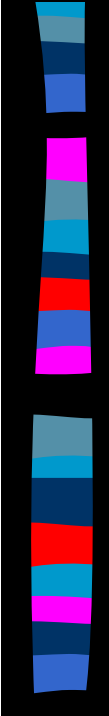
## C\* Communication Functions

### ■ Communication Functions allow you to

- Send values of parallel variable elements to other elements of the same shape.
- Send values of parallel variable elements of one shape to elements of an other shape.
- Perform different kinds of computation on values while sending them to elements of the same or different shape.
- Send data from parallel variable elements to a scalar variable, and from a scalar variable to a parallel variable element.
- Send data from a parallel variable to a scalar array, or from an array to a parallel variable.

2002/09/13

100

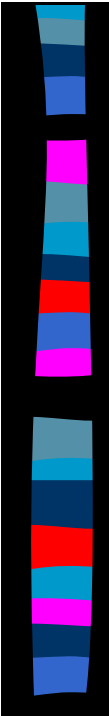


## Two Kinds of Communication

- Grad Communication
  - from\_grad, from\_grad\_dim
  - from\_torus, from\_torus\_dim
  - to\_grad, to\_grad\_dim
  - to\_torus, to\_torus\_dim
  - Pcoord
- General Communication
  - make\_send\_address
  - send, get
  - read\_from\_position, read\_from\_pvar
  - write\_to\_position, write\_to\_pvar
  - make\_multi\_coord

2002/09/13

101



## Communication and Computation

- scan
- spread, copy\_spread
- multispread, copy\_multispread
- enumerate
- rank
- reduce, copy\_reduce
- global

2002/09/13

102

# Grad Communication

## ■ Aspects of Grad Communication

- axis, direction, distance, border behavior, behavior of inactive positions

| Function       | Multiple Axes? | Wrapping? | Get or Send? |
|----------------|----------------|-----------|--------------|
| from_grad      | ✓              | ✗         | Get          |
| from_grad_dim  | ✗              | ✗         | Get          |
| from_torus     | ✓              | ✓         | Get          |
| from_torus_dim | ✗              | ✓         | Get          |
| to_grad        | ✓              | ✗         | Send         |
| to_grad_dim    | ✗              | ✗         | Send         |
| to_torus       | ✓              | ✓         | Send         |
| to_torus_dim   | ✗              | ✓         | Send         |

2002/09/13

103

## The from\_grad\_dim Function

**type:current** from\_grad\_dim (

**type:current** \*sourcep, **type:current** value, **int** axis, **int** distance);

);

dest = from\_grad\_dim(&source, fill, 0, -1);

|    |    |    |    |
|----|----|----|----|
| 10 | 11 | 12 | 13 |
| 20 | 22 | 22 | 23 |
| 30 | 31 | 32 | 33 |
| 40 | 41 | 42 | 43 |

source

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 10 | 11 | 12 | 13 |
| 20 | 21 | 22 | 23 |
| 30 | 31 | 32 | 33 |

dest

|   |   |   |   |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
| 1 | 2 | 3 | 4 |
| 1 | 2 | 3 | 4 |
| 1 | 2 | 3 | 4 |

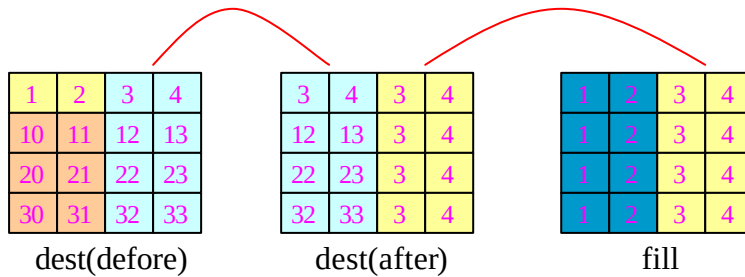
fill

2002/09/13

104

## The from\_grad\_dim Function (Cont.)

```
dest = from_grad_dim(&dest, fill, 1, 2);
```

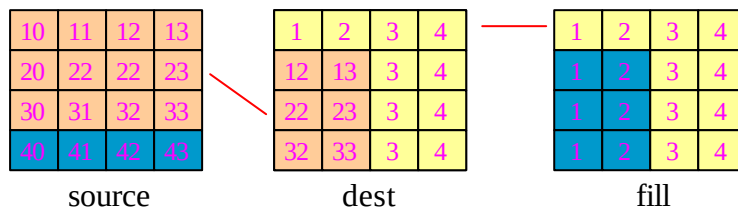


2002/09/13

105

## The from\_grad Function

```
type:current from_grad (
 type:current *sourcep, type:current value,
 int distance_along_axis_0,...);
);
dest = from_grad_dim(&source, fill, -1, 2);
```

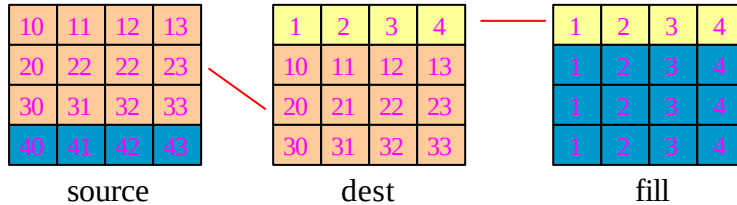


2002/09/13

106

## The to\_grad\_dim Function

```
void to_grad_dim (
 type:current *destp, type:current source, type:current *valuep,
 int axis, int distance);
);
to_grad_dim(&dest, source, &fill, 0, 1);
```

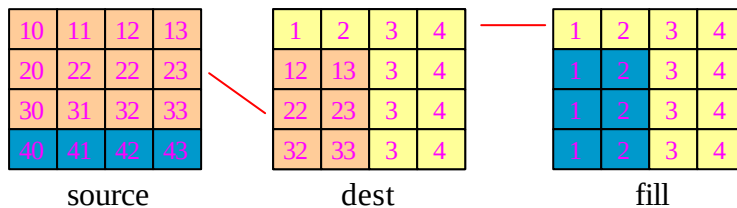


2002/09/13

107

## The to\_grad Function

```
void to_grad (
 type:current *destp, type:current source, type:current *valuep,
 int distance_along_axis_0,...);
);
to_grad (&dest, source, fill, 1, -2);
```

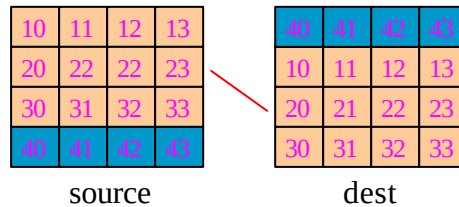


2002/09/13

108

## The from\_torus\_dim Function

```
type:current from_torus_dim (
 type:current *sourcep, int axis, int distance);
);
dest = from_torus_dim(&source, 0, -1);
```

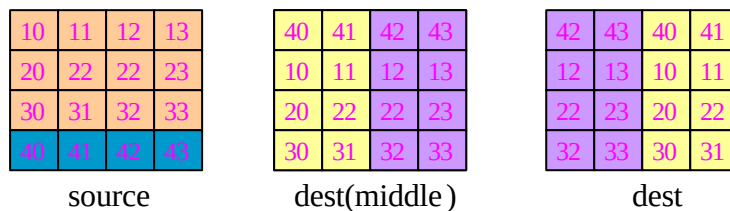


2002/09/13

109

## The from\_torus Function

```
type:current from_torus (
 type:current *sourcep, int distance_along_axis_0,...);
);
dest = from_torus_dim(&source, -1, 2);
```



2002/09/13

110

## The to\_torus\_dim Function

```
void to_torus_dim (
 type:current *destp, type:current source, int axis, int distance);
);
to_torus_dim(&dest, source, 0, -1);
```

|    |    |    |    |
|----|----|----|----|
| 10 | 11 | 12 | 13 |
| 20 | 22 | 22 | 23 |
| 30 | 31 | 32 | 33 |
| 40 | 41 | 42 | 43 |

source

|    |    |    |    |
|----|----|----|----|
| 20 | 22 | 22 | 23 |
| 30 | 31 | 32 | 33 |
| 40 | 41 | 42 | 43 |
| 10 | 11 | 12 | 13 |

dest

2002/09/13

111

## The to\_torus Function

```
void to_torus (
 type:current *destp, type:current source,
 int distance_along_axis_0,...);
);
to_torus (&dest, source, -1, 2);
```

|    |    |    |    |
|----|----|----|----|
| 10 | 11 | 12 | 13 |
| 20 | 22 | 22 | 23 |
| 30 | 31 | 32 | 33 |
| 40 | 41 | 42 | 43 |

source

|    |    |    |    |
|----|----|----|----|
| 20 | 22 | 22 | 23 |
| 30 | 31 | 32 | 33 |
| 40 | 41 | 42 | 43 |
| 10 | 11 | 12 | 13 |

dest(middle)

|    |    |    |    |
|----|----|----|----|
| 22 | 23 | 20 | 22 |
| 32 | 33 | 30 | 31 |
| 42 | 43 | 40 | 41 |
| 12 | 13 | 10 | 11 |

dest

2002/09/13

112

## The scan Function

```
type:current scan (
 type:current source, int axis, CMC_combiner_t combiner,
 CMC_communication_direction_t direction,
 CMC_segment_mode_t smode, bool:current *sbitp,
 CMC_scan_inclusion_t inclusion);
);
running_total = scan(data, 0, CMC_combiner_add, CMC_upward,
 CMC_none, CMC_no_field, CMC_inclusive);
```

|      | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|------|---|---|---|---|---|---|---|---|
| data | 4 | 7 | 9 | 5 | 3 | 5 | 9 | 6 |

|               |   |    |    |    |    |    |    |    |
|---------------|---|----|----|----|----|----|----|----|
| running_total | 4 | 11 | 20 | 25 | 28 | 33 | 42 | 48 |
|---------------|---|----|----|----|----|----|----|----|

2002/09/13

113

## The reduce Function

```
void reduce (
 type:current *destp, type:current source, int axis,
 CMC_combiner_t combiner,
 int to_coord);
);
reduce(&max, data, 0, CMC_combiner_max, 0);
```

|      | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|------|---|---|---|---|---|---|---|---|
| data | 4 | 7 | 9 | 5 | 3 | 5 | 9 | 6 |

|     |   |  |  |  |  |  |  |  |
|-----|---|--|--|--|--|--|--|--|
| max | 9 |  |  |  |  |  |  |  |
|-----|---|--|--|--|--|--|--|--|

2002/09/13

114

## The copy\_reduce Function

```
void copy_reduce (
 type:current *destp, type:current source, int axis,
 int to_coord, int from_coord);
);
copy_reduce(©, data, 0, 3, 0);
```

|    |    |    |    |
|----|----|----|----|
| 10 | 11 | 12 | 13 |
| 20 | 22 | 22 | 23 |
| 30 | 31 | 32 | 33 |
| 40 | 41 | 42 | 43 |

data

|    |    |    |    |
|----|----|----|----|
|    |    |    |    |
|    |    |    |    |
|    |    |    |    |
| 10 | 11 | 12 | 13 |

copy

2002/09/13

115

## The spread Function

```
type:current spread (
 type:current source, int axis, CMC_combiner_t combiner);
);
total = spread(data, 1, CMC_combiner_add);
```

|    |    |    |    |
|----|----|----|----|
| 0  | 1  | 2  | 3  |
| 10 | 11 | 12 | 13 |
| 20 | 21 | 22 | 23 |

data

|    |    |    |    |
|----|----|----|----|
| 6  | 6  | 6  | 6  |
| 46 | 46 | 46 | 46 |
| 86 | 86 | 86 | 86 |

total

2002/09/13

116

## The copy\_spread Function

```
type:current copy_spread (
 type:current source, int axis, int coordinate);
);
copy = spread(data, 1, 1);
```

|    |    |    |    |
|----|----|----|----|
| 0  | 1  | 2  | 3  |
| 10 | 11 | 12 | 13 |
| 20 | 21 | 22 | 23 |

data

|    |    |    |    |
|----|----|----|----|
| 1  | 1  | 1  | 1  |
| 11 | 11 | 11 | 11 |
| 21 | 21 | 21 | 21 |

copy

2002/09/13

117

## The enumerate Function

```
unsigned int:current enumerate (
 int axis, CMC_communication_direction_t direction,
 CMC_scan_inclusion_t inclusion, CMC_segment_mode_t smode,
 bool:current *sbitp);
);
number = enumerate(0, CMC_upward, CMCC_exclusive, CMC_none,
 CMC_no_field);
```

|        |   |   |   |   |   |   |   |   |
|--------|---|---|---|---|---|---|---|---|
|        | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| number | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

2002/09/13

118

# The rank Function

```
unsigned int:current rank (
 type:current source, int axis,
 CMC_communication_direction_t direction,
 CMC_segment_mode_t smode,
 bool:current *sbitp
);
data_rank = rank(data, 0, CMC_upward, CMCC_none, CMC_no_field);
```

|           | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|-----------|---|---|---|---|---|---|---|---|
| data      | 4 | 7 | 9 | 5 | 3 | 5 | 9 | 6 |
| data_rank | 1 | 5 | 6 | 2 | 0 | 3 | 7 | 4 |